Proyecto A, DA G2 2026-II

Samuel Restrepo Ramirez
COD: 0000034481
Universidad de Caldas
Programa: Ingeniería de Sistemas y Computación
Séptimo semestre.

Bloque 0: Problemas de Optimización

1. Problema vs Instancia

↓ La pregunta general.

↓ Un caso específico

Ejemplo El problema puede ser: Dado un arreglo, ordenalo.
La instancia: el arreglo A=[2,3,8,1]

Un algoritmo resuelve el problema para cualquier instancia.

2. Las 4 piezas de todo P.O.

1. **Datos**: La entrada, lo que me dan
2. **Variables de decisión**: ¿Qué decido yo? Lo que el algoritmo entrega
3. **Restricciones**: qué reglas se tienen
4. **Funcion Objetivo**: Con qué número mido que tan buena es la decisión.

Solucion factible

Cualquier solución que compla las restricciones por mala que sea

Solución óptima

La solución que mejora al máximo el valor de la función objetivo. (Pueden ser varias)

Utilidad: Esta es la función objetivo del proyecto. Le da un "puntaje" a la solución. Por eso hay que *Maximizar la utilidad* → buscar la solución factible con el puntaje más alto.

EJEMPLO:

Problema de la mochita:

Datos n (cantidad objetos), los pesos, los valores la capacidad.

Variables a quienes llevo y a quienes no (0,1)

Restriccion: que la suma de los pesos, no supere la capacidad.

Objetivo: Maximizar la suma de los valores.

- la instancia seria una entrada especifica de los datos

EJERCICIO:

- H horas para repartir
- M materias
- $g_j(h)$: cuanto sube la nota si le dedico h horas para cada materia j.

4 piezas:

Datos: las horas disponibles y las materias

Variables: cuantas horas van para cada j (materia)

restricciones: $\sum x_j \cdot h_j \leq H$ la suma de las horas debe ser menor o igual a las disponibles

Funcion Objetivo: Maximizar la nota

Una Instancia:

H=3 M: A ^ B

$g_j(h)$:

M ↓ \ h →	0	1	2	3
A	0	4	6	7
B	0	3	4	5

Sol factible: Ah=1 ^ Bh=2

No factible Ah=2 ^ Bh=2

Def. formales:

vars: h_j F.O. → $Max(\sum g_j(h_j))$

res: $\sum h_j \leq H$

PROPIEDAD FUNCIÓN CRECIENTE:

Usando el ejemplo anterior, si g(h) nunca baja, siempre hay una solución óptima que usa las H horas enteras.

- Por eso, "=" y "≤" dan el mismo valor
- Si la restricción es "=" y g(h) baja, obliga a tomar una solución peor.

U_i: En el enunciado dice que U_i es creciente → misma propiedad.

SALIDA BLOQUE 6:

PARTE 1: ($P_{0.}$ real)

- Entradas: N nodos, B presupuesto, U-i( ) (Datos)
- Variables: x_i, siendo i cada nodo ↳ cuanto se le asigna a c/u
- Restricciones: $\sum_{i=0}^{n} x_i = B$ → lo entregado sea igual al presupuesto
- Objetivo: $Max(\sum U_i(x_i))$ → Maximizar utilidad

Problema real PARTE 2

hay N nodos candidatos. f recibe un conjunto de nodos y devuelve la utilidad conjunta.

→ aporta más meter un nodo a un Conj. pequeño

→ elegir un conjunto $\leq K$ que maximice f

Entrada:

- N nodos $\in V$
- $V = \{1, \dots, N\}$
- $K \in \mathbb{N}^+$ $K < N$
- ORACULO: $f: 2^V, \forall A \subseteq B \rightarrow f(A) \leq f(B)$ y submodular.

Var. decision

$S \subseteq V$

Restriccion

$|S| \leq K \quad \vee \quad |S| = K$

Objetivo

Maximizar $f(S)$

BLOQUE 1: GANANCIA MARGINAL Y CONCAVIDAD

Ganancia Marginal:

Es lo que aporta la siguiente unidad de presupuesto.

Si un nodo tiene X unidades, su ganancia marginal es: $\Delta(x) = u(x+1) - u(x)$

EN VERSION DISCRETA

Creciente:

- Todas las Δ son ≥ 0
- Ninguna unidad resta

Cóncava:

- Las Δ NO aumentan.
- Cada Δ aporta menos o lo mismo.

Separable:

Lo que le doy a un nodo, no cambia nada de otros nodos, son independientes.

$U_1(X_1) + \dots + U_n(X_n)$

→ Cada U depende SOLO de su X.

EJERCICIO

1 Un nodo tiene $U = 0, 6, 10, 12, 13$

$\Delta = 6, 4, 2, 1$

R\ Creciente y Cóncava → $\{\forall \Delta \geq 0\} \wedge \{6 < 4 < 2 < 1\}$

2 B = 4, dos nodos P y Q

Marginales: P = 9, 5, 2, 1

Q = 7, 6, 1, 0

VORAZ:

VUELTA	Δ ACTUAL P	Δ ACTUAL Q	VA A	U. ACUMUL.
1	9	7	P	9
2	5	7	Q	16
3	5	6	Q	22
4	5	1	P	27

SUBMODULARIDAD:
BLOQUE 2

Concepto: En la PARTE 2 no se reparten unidades:

→ Se eligen nodos.

la ganancia marginal:

de agregar un nodo 'v' dado que ya tengo el subconjunto S:

$$\Delta(v \mid S) = f(S \cup \{v\}) - f(S)$$

Ganancia M = lo que tendría si meto v − lo que ya tengo

... Y **SUBMODULAR:**

Entre más tengo, menos aporta lo nuevo.

Para $\forall A \subseteq B$ se diría así:

$$\Delta(v \mid A) \geqslant \Delta(v \mid B)$$

La ganancia de meter 'v' en el chiquito ⩾ que meterlo en el grande.

↝ Es como decir las Δ no aumentan pero con conjuntos.

Un EJEMPLO:

- Hay unos nodos llamados: 1, 2 y 3 (los elijo)
- hay unos elementos: a, b, c, d, e que los nodos contienen.

1: {a, b, c} 2: {c, d} 3: {d, e}

S → es el equipo de nodos que se elegirán

EJ: $S = \{1\}$ S tiene al nodo 1

f(S) → ¿cuantas letras DISTINTAS cubre el equipo S?

si agregamos el nodo 2 mientras $S = \{\emptyset\}$ ...

$$\Delta(2 \mid \emptyset) = f(\{2\}) - f(\emptyset) = 2 - 0 = 2$$

Nuevo − Viejo; cubre c y d

pero si agregamos el 2, cuando ya estaba el 1:

$$\Delta(2 \mid \{1\}) = f(\{1,2\}) - f(\{1\}) = 4 - 3 = 1$$

|a, b, c, d| − |a, b, c|

Conclusión: Agregar el N2 aporta menos cuando N1 ya estaba, porque 'c' ya estaba cubierto

↓

Eso es la SUBMODULARIDAD

VERSION INFORME:

Sea cub(S) el conjunto de letras que cubre S y L(v) las letras del nodo v. Entonces $\Delta(v \mid S) = |L(v) \text{ sin } cub(S)|$

Monótona: es la cantidad de elementos de un conjunto $\Delta \geqslant 0$

Submodular: si $A \subseteq B$, todo nodo de A está en B, así que $cub(A) \subseteq cub(B)$. Por eso $L(v) \text{ sin } cub(B) \subseteq L(v) \text{ sin } cub(A)$, y $\Delta(v \mid B) \leqslant \Delta(v \mid A)$

EJ: VORAZ

Vuelta	Eg. Antes	Δ(1)	Δ(2)	Δ(3)	Elegido
1	∅	3	2	2	1
2	{1}	-	1	2	3

→ ¡VORAZ encontró el óptimo! Pero, no siempre lo hace lo Logra Aprox. 63%.

VORAZ en Parte 1 - 100%

VORAZ en Parte 2 - 63% (1-1/e)

B

BLOQUE 4:

P, NP Y REDUCCIONES

Conceptos:

① Problema de decisión: ¿SÍ o NO?

Parte 2 → Es de optimización, pero tiene un componente de decisión:

"¿Existe un equipo S con $|S| \leq K$ y $f(S) \geq t$?"

- t es una meta.

② P: Son los problemas que se pueden resolver con T(n) polinomiales.

Parte 1 → $\in O(B \cdot \lg N)$

③ NP: Problemas en los que:

si alguien me entrega una solución candidata:

la puedo VERIFICAR en T(n) polinomial.

④ NP-Difícil: Es un problema AL MENOS tan difícil como cualquiera de NP. ~~en tiempo~~

Si lo resolviera en T(n) polinomial, resolvería todos los NP en polinomial. → Nadie sabe hacerlo !!

⑤ Reducción A a B: Es una traducción.

Tomamos cualquier entrada de A (y en T(n) polinom.) la convertimos en una entrada de B tal que:

- La respuesta de B es la misma que la de A

↓

Si eso se puede:

- B es AL MENOS tan difícil como A

Receta de PRUEBA NP-DUREZA POR REDUCCIÓN

1 ORIGEN
El problema que ya se sabe NP-difícil en su versión de decisión.

2 CONSTRUCCIÓN
El como pasar de la entrada origen a mi problema.

3 TIEMPO
la traducción es $T(n) \in \Theta$(polinomial)

4 VALIDEZ
La entrada construida cumple las hipótesis de mi problema.
↳ f monótona y submodular

5 EQUIVALENCIA

Si en Origen ↔ Si en mi problema

Bidireccional

Entonces, CM → selección nodos

1) Origen en versión decisión:

CM recibe un universo de U elementos (conjuntos) $C_1, C_2, \dots, C_m \subseteq U$, un entero K y meta t

¿Existen K conjuntos de la colección cuya unión tenga al menos t elementos?

2) Construcción: origen → mio.

FORMAL

Sea $I = (U, C, K, t)$ una instancia de cobertura máxima, donde U es un universo finito de elementos, $C = \{C_1, \dots, C_m\}$ es una colección de subconjuntos de U, $K \in \mathbb{N}$ es el número máximo de conjuntos que se pueden elegir y $t \in \mathbb{N}$ es la meta de cobertura.

A partir de I se construye $I' = (V, f, K', t')$ del problema selección de nodos-P2.

a) Nodos: $N = m$ y $V = \{1, \dots, m\}$

b) f de utilidad: $\forall S \subseteq V$, $f(S) = |\bigcup_{i \in S} C_i|$ es decir, el número de elementos distintos de U que cubren ~~todos~~ los conjuntos de los nodos elegidos

c) Parámetros: $K' = K$ y $t' = t$

$I' \rightarrow$ ¿existe $S \subseteq V$ con $|S| \leq K'$ y $f(S) \geq t'$?

3) Construcción polinomial

Una reducción sólo sirve si traducir la instancia de CM a la parte 2 es barato. Si traducir ya fuera exponencial, no probaría nada sobre la dificultad. Por eso, se cuenta lo que vale construir cada pieza.

Pieza	Qué se hace	Costo en Notación
K' y t'	Copiar dos números	$\Theta(1)$
$V = \{1, \dots, m\}$	Un nodo por conjunto	$O(m)$
Conjuntos C_i	Guardarlos tal cual	$O(\sum C_i)$ ⇒ la suma de los tamaños de todos los Conjunt
función f	Se entrega como un oráculo: una función que, cuando le das un S, calcula la unión en ese momento	Construirlo: $O(1)$ C/llamada: $O(\sum C_i)$ WORST

Porqué un oráculo y no una tabla: Una tabla necesitaría una fila por cada subconjunto posible de nodos, y hay 2^m conjuntos.

4) lo construido es una instancia válida de la parte 2.

la parte 2 solo acepta funciones f monótonas y submodulares. Si la f de cobertura no lo fuera, no estaríamos reduciendo nuestro problema, sino a otro. Hay que probar las 2 propiedades entonces.

Notación: $\Delta(v|S) = f(S \cup \{v\}) - f(S)$

Lo que aporta el recien llegado v dado que S ya está

$\Delta(v|S)$ = Cuantas letras de v no estaban cubiertas por el equipo.

Monótona (agregar nodos no baja utilidad)

$\Delta(v|S)$ cuenta letras y un conteo nunca es negativo, así que

$$\Delta \geq 0$$

Submodular: (el recién llegado aporta menos, o lo mismo, a un equipo grande que a uno pequeño)

Si $A \subseteq B$ lo, todo lo que cubre A también lo cubre B. Entonces, las letras de v que siguen sin cubrir frente a B son, como mucho, las que siguen sin cubrir frente a A; por eso:

$$\Delta(v|B) \leq \Delta(v|A)$$

P. 5) Bidireccionalidad

CM: Sí ⟷ Parte 2: Sí

FORMAL: → Supongamos que existen $C_{i_1}, \dots, C_{i_k}$ cuya unión tiene al menos t elementos.

Sea $S = \{i_1, \dots, i_k\} \subseteq V$. ENTONCES:

$|S| = k' = k$, y por la definición de f, $f(S) = |C_{i_1} \cup \dots \cup C_{i_k}| \geq t = t'$. Luego, la instancia construida responde SI.

← Sea $S = \{v_1, \dots, v_r\} \subseteq V$ con $r \leq k'$ y $f(S) \geq t'$. Sea $j = k - r$. como $k < N = m$, el conjunto $V \setminus S$ tiene al menos j nodos; sea $W = \{w_1, \dots, w_j\} \subseteq V \setminus S$.

Considérese la colección $S = \{C_{v_1}, \dots, C_{v_r}, \dots, C_{w_j}\}$ que tiene $r + j = k$ conjuntos. Por definición de f, la unión de S tiene $f(S \cup W)$ elementos, y como f es monótona y $S \subseteq S \cup W$

$$f(S \cup W) \geq f(S) \geq t' = t$$

CM responde SI

CASO PRUEBA: (Por el monitor)

Campo

$K = 5$, $B = 5$

Coeficiente utilidad

Grafo completo con 105 aristas

Presupuesto total nodo = 18

3 rondas; costos de participación con presupuesto de 24 → 15/5 = 3

En sol. base.

Ronda.

Es a en: $U_i(x) = a \cdot \ln(1+x)$ (sup)

f supuesto

Tope C_i

Variantes b y a. Solo se reportan.

* Cada nodo w vale su coeficiente. Si w participa, su info entra completa al modelo. Si no, entra a través de cada nodo elegido, con la prob. que da el peso de la arista.

Entonces: (Decisiones según la entrada):

- Para la utilidad conjunta de Parte 2 uso el grafo. Cada nodo aporta su 'coeficiente_utilidad' según qué tan representado queda por los nodos elegidos:
  - Completo si participa y si no, a través del peso influencia de cada elegido.
- Para Parte 1: $U(x) = \text{coeficiente} \cdot \ln(1+x)$

CONCEPTOS IMPORTANTES:

Monotonía:

Cuando entra un nodo cada probabilidad de 'sin representar' se multiplica por un número entre 0 y 1, entonces f nunca baja.

Submodularidad (en este caso):

El aporte de v depende de esas probabilidades. Y esas, solo bajan cuando el equipo crece.

Costo:

Son igual $\Theta(N \cdot K)$ llamadas y cada una cuesta $O(1 + \text{grado})$, y con el grafo completo:

$\rightarrow O(K \cdot N^2)$

Tope:

El nodo que llega a su tope sale del montículo.

DISEÑO DE SOLUCION BASE P1, P2 y RONDA COMP.

Parte 1 Reparto del presupuesto.

Qué resuelve: El equipo YA está elegido; n nodos. Hay B unidades para repartir. Cada nodo i tiene una utilidad $U_i(x)$, que es lo que aporta si recibe x unidades, y un tope C_i, que es lo máximo que puede recibir.

Meta: Repartir para que la suma de las unidades sea la mayor posible.

$n = |S|$: Nro de nodos en el equipo

B = Unidades para repartir en la ronda.

$\Delta_i(x)$ lo que vale la SIGUIENTE

C_i tope del nodo

* recordar que es cóncava.

Subasta: Unidad a unidad.

Las unidades se reparten 1 a 1, y se la lleva el nodo que más valor da en eje momento.

fonciona porque: la concavidad hace que los aportes se vayan reduciendo a medida que se agregan unidades a un nodo: Esto, organiza de mayor a menor.

Monticulo:

En cada vuelta hay que encontrar la mayor de n ganancias:

→ Un montículo de máximos es un árbol de madre e hijos, donde la madre vale AL MENOS lo que sus hijos.

→ SACAR o METER resta lo altura

PSEUDOCODIGO:

```
Repartir_presupuestos(S, B, u, c)
1 para cada i ∈ S hacer x[i] ← 0
2 H ← BUILD-MAX-HEAP con (Δi(0), -i)
3 para unidad ← 1 hasta B, mientras H > 0:
4     (δ, -i) ← EXTRAER-MAXIMO
5     x[i] ← x[i] + 1
6     if x[i] < Ci: INSERTAR(H, (Δi(x[i])), -i)
7     TRAZA(unidad, i, δ)
8 devolver ↑ x, ∑ ui(x[i])
```

1 Nadie tiene nada aún.
2 Arma el montículo con la 1ra unidad de c/nodo (máx → raiz)
3 Da una vuelta por c/unidad de B y se reparten todas
4 Saca la mejor gancia posible (voraz)
5 Le da la unidad a su nodo x[i]
6 Si al nodo le cabe más, vuelve a su fila con la sgte.
7 Anota la traza para evidenciar.
8 Entrega el reparto y su utilidad

¿Por qué es óptimo?

Intuitivamente, si un reparto le da a i una unidad que vale menos que j se quedó sin recibir, pasar esa unidad de i a j no baja

el total. Repitiendo eso se llega al voraz sin perder nada

COSTO

- BUILD-MAX-HEAP es $O(n)$
- Por vuelta: Cada una hace una extracción y máximo una inserción, c/u en $O(\lg n)$ y máximo hay B vueltas.
- Entregar reparto y utilidad cuesta $O(n)$

$T(n, B) = O(n + B \cdot \lg n)$

Parte 2

Selección del equipo

Resuelve que hay N candidatos y cupo para K. $f(S)$ es la utilidad del equipo S

Meta: Elegir el equipo de, máximo, K nodos con la mayor f posible.

- f depende de cómo se complementan los nodos (recordar Submodular)
- Monótona: agregar un nodo nunca resta

No es exacto (esta solución) porque el problema es NP-difícil (ya demostrado con CM). El exacto por fuerza bruta tendría que probar todos los equipos de tamaño K (= K por ser creciente), y ese número crece exponencialmente cuando K crece con N.

⇝ Entonces se usa un voraz con garantía.

Oráculo:

f se entrega como un procedimiento que, dado S, devuelve $f(S)$. NO como tabla porque esta tendría una Fila por subconjunto: 2^N.

⇝ El costo se cuenta como llamadas al oráculo.

* Para no recalcular desde ∅, el oráculo lleva un estado del equipo

Las utilidades conjuntas:

Cobertura: C/nodo cubre unos elementos. $f(S)$ es cuántos elementos DIFERENTES cubre.

Influencia

- Cada nodo w vale a_w, su coeficiente
- Si w está en el equipo la info entra toda

- Si no, el nodo elegido la transmite con probabilidad p_vw (Peso de su arista)
- q_w es la probabilidad de que la info de w siga sin entrar
- $f(S) = \sum a_w \cdot (1 - q_w)$: cuanta info se espera que entre.

PSEUDOCODIGO

```
Seleccionar_nodos(V, k, f){
1 S ← ∅ ; valorS ← f(∅)
2 para paso ← 1 hasta k:
3     mejor ← NIL ; mejorΔ ← -∞
4     para cada v en V \ S:
5         Δ ← f(S ∪ {v}) - valorS
6         si Δ > mejorΔ:
7             mejor ← v ; mejorΔ ← Δ
8     S ← S ∪ {mejor} ; valorS ← valorS + mejorΔ
9     TRAZA(paso, aportes evaluados, mejor, mejorΔ,
            valorS)
10 ↑ S, valorS
}
```


1 Punto de partida → equipo vacío.
2 K pasos, un recien llegado por c/paso.
(Recordar que por monotonía se llena K)
3 ~
4 Recorre los que todavía no están.
5 Calcula lo que aportaría cada candidato. (OP a contar)
6 se queda con uno estrictamente mejor
8 Lo agrega y suma su aporte.

COSTO

En el paso p el equipo ya tiene (p-1) nodos, entonces se evaluan N-p+1 candidatos:

$$T(N,K) = N + (N-1) + \cdots + (N-K+1) = K \cdot N - \frac{K(K-1)}{2}$$

Por enuerro → $\Theta(N \cdot K)$

- Ningún ciclo termina antes de tiempo, el nro de llamadas es el mismo para toda entrada

TODO JUNTO

archivo.json

↓ entrada.py: lee y revisa la forma

Verificar la entrada

- $1 \leq K < N$, $B \geq 0$
- ids unicos
- ui concavas
- pesos en $[0, 1]$

} si algo falla, NO se corre y la traza dice por qué

↓

Parte 2. Seleccionar los nodos (V, f, K) — de acá sale el equipo S de K nodos.

↓

Parte 1. Repartir el presupuesto (S, B, u, c) — sale x: cuanto recibe c/nodo

↓

Verificar la salida

- Traza
- Tiempos
  - mediana
- informativos

¿Por qué en ese orden?

Primero se necesita saber a QUIENES se les va a repartir el presupuesto, y luego repartirlo. El reparto depende de la selección, en cambio, la selección es independiente del reparto.

Son realmente 2 objetivos distintos

- La selección maximiza f → lo que hemos llamado cobertura o influencia
- El reparto maximiza $\sum u_i$.

SON DIFERENTES pero hacen merge.

hasta aquí 1ra entrega · Samuel Restrepo Ramírez